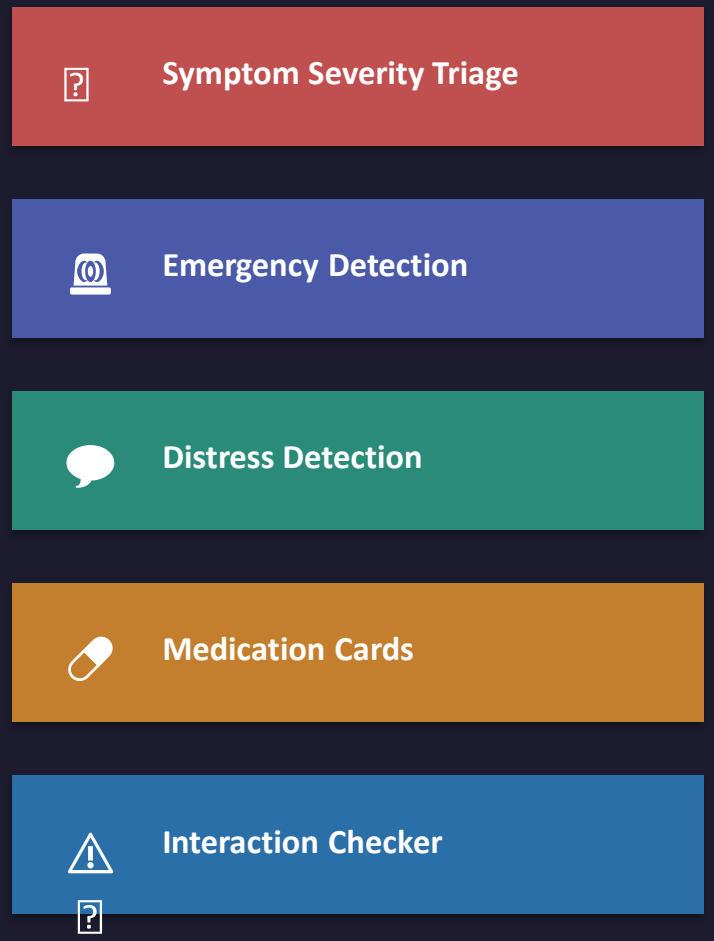
Medical Agentic Framework

NLP Group Project • BUAN6342

The University of Texas at Dallas | May 2026

Abin Roy, Rohit S. Patala, Seereen Zoubi

An Agentic RAG System for Medical Inquiry



USE CASE

What Problem Are We Solving?

Patients often struggle to interpret their symptoms, understand medications, and locate nearby healthcare providers quickly. This project addresses that gap by building a full-stack Medical Agentic Framework — a cloud-native AI concierge that handles symptom triage, medication guidance, and doctor discovery in a single conversational interface. The system uses Retrieval-Augmented Generation (RAG) grounded in curated medical datasets to produce accurate, context-aware responses — eliminating the hallucination risk of general-purpose LLMs.

Problem

Patients lack fast, reliable symptom & medication guidance

Approach

Agentic RAG with Dialogflow CX, Vertex AI & Gemini LLM

Outcome

Live deployed chatbot on Cloud Run with 5 safety features

METHODOLOGY

Corpus & Datasets

DS1

Symptom–Disease Mapping

Source: Kaggle — itachi / Pranay Patil

133 symptoms with severity weights 1–7 · used directly for triage scoring

DS2

Augmented Disease Dataset

Source: Kaggle — dhivyeshrk

773 diseases · 377 binary symptom columns · 246,945 rows — classifier foundation

DS3

Symptom Descriptions

Source: symptom_Description.csv (bundled)

Plain-English disease descriptions ingested into the Vertex AI knowledge base

DS4

Symptom Precautions

Source: symptom_precaution.csv (bundled)

Up to 4 recommended precautions per disease for agent response grounding

DS5

Hospital Directory

Source: CMS Hospital General Information (CMS.gov via Kaggle)

4,818 US hospitals · real addresses, phone numbers & CMS star ratings · powers nationwide city-based lookup

DS6

Medication Data

Source: Curated CSV

Drug details, dosages & contraindications (e.g. Ibuprofen / Ulcer logic)

METHODOLOGY

Data Preparation & Cleansing

merge_code.py — 6-step pipeline

Load Raw CSVs

1

Three separate files: dataset.csv, symptom_Description.csv, symptom_precaution.csv

Symptom Consolidation

2

Symptom_1…Symptom_N pivoted into a single All_Symptoms field via lambda + dropna + str.strip()

Precaution Consolidation

3

Precaution_1…Precaution_4 merged into All_Precautions using the same pivot pattern

Deduplication

4

drop_duplicates(subset=['Disease']) ensures one canonical row per disease

Multi-Dataset Merge

5

Three-way left join on 'Disease' key: symptoms + descriptions + precautions

Export to Vertex AI Format

6

Structured TXT (vertex_knowledge_base.txt) with DISEASE / DESCRIPTION / SYMPTOMS / PRECAUTIONS blocks for GCS

Code Overview

Key logic only — no full code dumps

merge_code.py

Load 3 CSVs with pandas.read_csv()

Pivot symptom & precaution columns into single field

Three-way left-join on 'Disease' key

Export structured .txt for Vertex AI ingestion

app.py — Streamlit Front-End

Session state: message history + UUID session_id

get_agent_response() calls detect_intent() via SDK

Renders chat with st.chat_message()

Extracts response_messages[0].text.text[0]

Feature Modules (app_enhanced.py)

severity_triage.py

Loads Symptom-severity.csv · tokenizes user message · sums weight per matched symptom · checks emergency combos first

distress_detection.py

Tiered scoring (×4/×2/×1.5) · 12 high-signal fear words, 22 moderate distress words, 19 regex urgency patterns · DistilBERT upgrade path

medication_card.py

Fuzzy-matches disease in agent reply (0.82 threshold) · background detect_intent call · caches by disease

interaction_checker.py

Regex scans ~50 drug names · accumulates session list · triggers background check when ≥2 drugs found

METHODOLOGY

Testing the Model

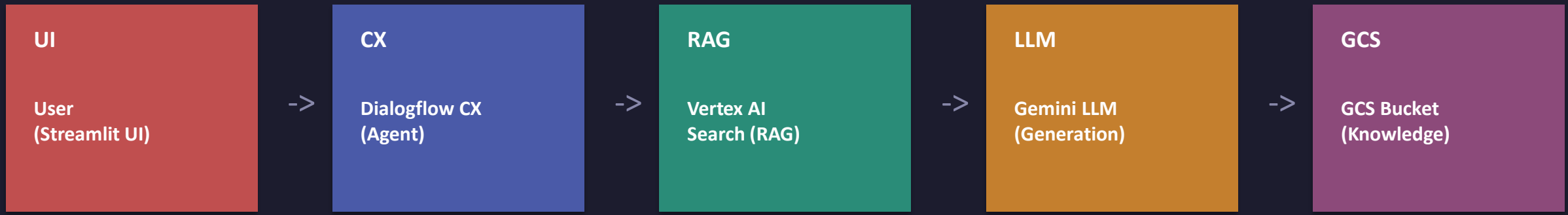
All cases verified against live modules · Features 4 & 5 require GCP credentials

User Message	Triage Result	Distress	✓
"I have a slight headache and itching."	Low (score: 4)	Low — no indicator	✓
"Fatigue, vomiting, stomach pain, fever."	Moderate (score: 21)	Low — no indicator	✓
"Scared, chest pain, cannot breathe!!!"	● Emergency	High (0.65)	✓
"Dizzy, joint pain and nausea."	Low (score: 12)	Low — no indicator	✓
"PLEASE HELP, passed out, now shaking"	● Emergency	High (0.80)	✓

⚡Features 4–6 (medication card, interaction checker, hospital lookup, disease precaution card) tested manually via live Dialogflow agent — see TEST_STATEMENTS.md.

SYSTEM ARCHITECTURE

End-to-End Request Flow



1 User types a message in Streamlit

2 get_agent_response() calls detect_intent() via Dialogflow CX SDK

3 Dialogflow CX queries Vertex AI Search for relevant medical context

4 Gemini synthesises the grounded response and returns it via Dialogflow CX

5 Streamlit renders reply; local modules run triage/distress checks; Gemini tag protocol triggers structured cards (hospital, medication, precaution, interaction)

Local Python Modules — no GCP changes needed

severity_triage.py · distress_detection.py

medication_card.py · disease_precaution_card.py

interaction_checker.py · hospital_provider_card.py

Gemini Tag Protocol:
[ADVANCE_CONDITION]
[SHOW_HOSPITALS:]
[SHOW_MEDICATIONS]
[SHOW_INTERACTION]

hospital_provider_card.py & disease_precaution_card.py

hospital_provider_card.py

Dataset: CMS Hospital General Information — 4,818 US hospitals
Addresses, phone numbers, CMS star ratings, emergency services flag
Triggered by Gemini tag [SHOW_HOSPITALS:<city>] — city lookup with prefix fallback
Sorted: emergency first, then CMS rating desc — session-cached per city
Graceful not-found card with Medicare Care Compare fallback link

disease_precaution_card.py

Covers 41 conditions from the Itachi symptom-disease dataset
Renders symptom list, precautions & AI-enriched description per disease
Triggered by [ADVANCE_CONDITION] — auto-fires after triage identifies condition
_PROFILES_LOWER alias map — case-insensitive condition name matching
Falls back to “See response above” for out-of-scope conditions

Gemini Tag Protocol

Gemini prepends structured tags that Python strips and acts on — enables new features without touching Dialogflow or GCP configuration.
[ADVANCE_CONDITION] [SHOW_HOSPITALS:<city>] [SHOW_MEDICATIONS] [SHOW_INTERACTION] [SHOW_CAPABILITIES]

CONCLUSION

Summary & Key Results

This project demonstrates the end-to-end design and deployment of a cloud-native Agentic RAG system for medical inquiry. Starting from raw CSV datasets, we engineered a structured knowledge base, indexed it in Vertex AI Search, and used Dialogflow CX with Gemini to produce responses grounded in domain-specific data — eliminating hallucination risks. Six safety and UX features (symptom severity triage with emergency detection, distress detection, proactive medication cards, drug interaction checking, disease precaution cards, and nationwide hospital lookup) were layered on top with zero GCP infrastructure changes, using a Gemini tag protocol that lets Python intercept structured tags and render rich data cards. Containerized with Docker and deployed on Cloud Run, the app is publicly accessible and scales automatically.

6	3	773	246K	Live ✓
Safety & UX Features	Datasets Merged	Diseases in KB	Training Rows	Cloud Run Deploy

REFERENCES

Sources & Tools

Datasets

Pranay Patil (itachi). Disease-Symptom Severity Dataset. Kaggle.
https://kaggle.com/datasets/itachi9604/disease-symptom-description-dataset

dhivyeshrk. Diseases and Symptoms Dataset. Kaggle.

https://kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset

symptom_Description.csv, symptom_precaution.csv

Bundled with Pranay Patil dataset.
The Devastator. Hospitals in the US (CMS Hospital General Information). Kaggle.
kaggle.com/datasets/thedevastator/hospitals-in-the-united-states-a-comprehensive-d
MGHOBASHY. Drug-Drug Interactions (191,542 pairs). Kaggle. [optional upgrade]
kaggle.com/datasets/mghobashy/drug-drug-interactions

Cloud & AI Platforms

Google Cloud Dialogflow CX

cloud.google.com/dialogflow/cx/docs

Google Vertex AI Search

cloud.google.com/vertex-ai/docs

Google Cloud Storage · Google Cloud Run

cloud.google.com/storage · cloud.google.com/run

Gemini 2.5 Flash (via Dialogflow CX backend)

Libraries & Tools

Streamlit — Python web UI

docs.streamlit.io

google-cloud-dialogflow-cx — Python SDK

pypi.org/project/google-cloud-dialogflow-cx/

pandas · Docker

pandas.pydata.org · docker.com

HuggingFace transformers / DistilBERT (optional)

huggingface.co/distilbert-base-uncased